A Game-Theoretic Analysis of MEV in Decentralised Exchanges

Final Report

Name	Roll Number
Sanoj S Vijendra	22B0916

April 19, 2026

Abstract

The report presents a comprehensive literature review on Maximal Extractable Value (MEV) and its game theoretic dimensions in decentralised exchange (DEX) ecosystems, followed by a rigorous formal model of the strategic game among users, miners/validators, and DEX protocols. We survey foundational and recent works on frontrunning, Priority Gas Auctions (PGAs), MEV driven consensus instability, and batch-auction-based mitigations. Also, report includes the simulation code behaviour and explanation. Click here for code.

Contents

1 Introduction **3**

2 Literature Review **3**
2.1 MEV: Origins and Measurement . . . 3
2.1.1 Pure Revenue Opportunities . . . 3
2.1.2 Priority Gas Auctions (PGAs) . . . 3
2.1.3 MEV and Consensus Instability . . . 4
2.2 Fair Ordering and MEV Mitigation . . . 4
2.2.1 Randomised Sequencing . . . 4
2.2.2 Proposer-Builder Separation (PBS) . . . 4
2.3 Batch Auctions as a Market Design Response . . . 5
2.4 Related Protocols and Extensions . . . 5
2.5 Summary of Literature Gaps . . . 5

3 Formal Game Theoretic Model **5**
3.1 Overview and Modelling Philosophy . . . 6
3.2 State Space . . . 7
3.3 Players . . . 7
3.3.1 Player 1: The User Set . . . 7
3.3.2 Player 2: The Miner/Validator . . . 7
3.3.3 Player 3: The DEX Protocol . . . 8
3.4 Strategy Spaces . . . 8
3.4.1 User Strategy Space . . . 8
3.4.2 Miner Strategy Space . . . 9
3.4.3 Sandwich Attack: Formal Specification . . . 9
3.5 Payoff Functions . . . 10
3.5.1 User Payoff . . . 10
3.5.2 Miner Payoff . . . 10
3.5.3 Protocol Payoff / Social Welfare . . . 11
3.6 Information Structure . . . 11
3.7 Equilibrium Concepts . . . 11
3.7.1 Nash Equilibrium . . . 11
3.7.2 Stackelberg Equilibrium . . . 12
3.7.3 Repeated Game and Cooperation . . . 12
3.8 Priority Gas Auction Sub-Game . . . 12
3.9 The Batch Auction Extension . . . 13

4 Comparative Analysis: Continuous vs. Batch Clearing **13**
4.1 Equilibrium Outcomes Under Continuous Trading . . . 13
4.2 Equilibrium Outcomes Under Batch Auctions . . . 14
4.3 Welfare Comparison . . . 14

5 Code Introduction **15**

6 Module Descriptions **15**
6.1 txn.py: Transaction . . . 15
6.2 dex.py: Automated Market Maker . . . 16

6.3 block.py: Block 17
6.4 blocktree.py: Block Tree 17
6.5 peer.py: Peer 18
6.6 pga.py — Priority Gas Auction 19
6.7 mempool.py — Mempool 20
6.8 blockchain.py: Blockchain 21
6.9 simulation.py — Event-Driven Simulation Engine 21

7 Interaction Between Modules **24**
7.1 Transaction Generation and PGA Formation 24
7.2 Block Assembly in Continuous Mode 24
7.3 Block Assembly in Batched Mode 24
7.4 Block Propagation 25

8 Simplifications and Limitations **25**

1 Introduction

Decentralised exchanges (DEXs) have emerged as a cornerstone of the Decentralised Finance (DeFi) ecosystem enabling non-custodial, trustless token swapping through on-chain smart contracts. Unlike their centralised counterparts, DEXs expose the full transaction pipeline from submission to final settlement to a public peer-to-peer mempool, creating rich informational asymmetries that rational actors exploit for profit.

The most consequential of these exploits fall under the umbrella of *Maximal Extractable Value* (MEV): the surplus value that miners or validators can capture by selectively reordering, inserting, or censoring transactions within a block. MEV manifests in several forms, most notably *frontrunning*, *backrunning*, and *sandwich attacks*, each of which directly reduces user welfare while enriching block producers and sophisticated arbitrage bots.

2 Literature Review

2.1 MEV: Origins and Measurement

The concept of Miner Extractable Value was formally introduced and empirically grounded by **Daian et al.** [1] in the landmark paper *Flash Boys 2.0: Frontrunning, Transaction Reordering, and Consensus Instability in Decentralised Exchanges*. Drawing on nine months of Ethereum mempool data collected from six geographically distributed nodes, the authors documented a thriving ecosystem of arbitrage bots exploiting price inefficiencies across multiple DEXs.

2.1.1 Pure Revenue Opportunities

Daian et al. identify a tractable sub class of MEV called *pure revenue opportunities*: atomic smart-contract transactions that execute multiple trades across one or more exchanges and yield positive profit *in every traded asset*, regardless of market movement. The all or nothing atomicity of Ethereum smart contracts is key, if any leg of the trade fails, the entire transaction reverts, eliminating execution risk.

The authors establish a lower bound of **over $6M USD** in pure revenue captured by bots on Ethereum across the observation period (July 2017–March 2019), with 1–10 ETH available for arbitrage on a typical day and occasional windfalls exceeding 100 ETH in a single block (e.g., block 7,029,147, where a single arbitrage trade generated 101.6 ETH, more than 33× the block reward).

2.1.2 Priority Gas Auctions (PGAs)

Because pure revenue opportunities are broadcast publicly via the peer-to-peer gossip network, competing bots race to be the first to exploit them. This race takes the form of a **Priority Gas Auction (PGA)** where bots submit the same transaction repeatedly with increasing gas prices, bidding for priority inclusion in the next block.

Key empirical observations from Daian et al. include:

- PGAs are continuous-time, partial-information games with *all-pay* characteristics. Even losing bids consume gas fees.

- Bots converge over time to a **minimum bid-raise strategy** (12.5%, the Parity client's enforced minimum increment), consistent with a cooperative Nash equilibrium.
- Latency is a critical competitive dimension, mean inter-bid times fell from ∼0.8 s in mid-2018 to ∼0.1 s by early 2019.
- A **grim-trigger cooperative equilibrium** emerges organically on-chain, where players alternate minimum-increment bids and threaten to raise to the maximum if the other defects.

2.1.3 MEV and Consensus Instability

Beyond user welfare concerns, Daian et al. demonstrate that high MEV creates systemic risks at the *consensus layer* such as:

- **Undercutting attacks**: A miner observing a block with large Order Optimisation (OO) fees can profitably fork that block, withholding some fees to attract competing miners to build on the fork. The authors show this is a *present* threat because OO fees already dominate block rewards in some blocks.
- **Time-bandit attacks**: A miner can rewrite blockchain history to retroactively capture MEV locked in past blocks. With DEX volumes peaking at $1.5B/month and a 51% attack on Ethereum estimated at $56M/month (as of 2019), the economics of such attacks are plausible in high-MEV regimes.

2.2 Fair Ordering and MEV Mitigation

2.2.1 Randomised Sequencing

Alipanahloo, Zhang, and Awosika [3] propose *fair ordering policies* for Layer-2 rollups that randomise transaction sequencing to reduce the predictability that MEV extraction relies upon. They introduce the notion of *fairness granularity*: the minimum resolution at which a sequencer can distinguish and reorder transactions. Their key finding is that coarser granularity (i.e., batching transactions over wider time windows before ordering) reduces MEV opportunities but also increases latency.

While this work provides formal fairness guarantees and quantifies the MEV-latency trade-off, it does not model the *strategic response* of users or miners to such a policy change which we are trying to address.

2.2.2 Proposer-Builder Separation (PBS)

The Ethereum research community has proposed **Proposer-Builder Separation** (PBS) as a structural remedy: specialised *block builders* compete to construct the most profitable block, while *proposers* (validators) simply select the highest-revenue block without seeing its contents. PBS partially democratises MEV revenue by routing it through an open auction, but critics note it may entrench builder oligopolies and does not eliminate user harm from frontrunning.

2.3 Batch Auctions as a Market Design Response

Budish, Cramton, and Shim [2] provide the theoretical foundation for batch auctions in the context of traditional financial markets. Their central insight is that continuous limit order book trading creates a *latency arms race* in which any finite speed advantage allows high frequency traders to exploit stale quotes, wasting social resources on latency investment and producing a market that is efficient only in a narrow technical sense.

The authors propose **frequent batch auctions** (FBAs) as a solution. Orders are collected over a short discrete time interval (e.g., 100 ms) and cleared simultaneously at a single uniform price. This batching eliminates the value of sub-interval latency advantages because all orders received within the same batch are treated symmetrically.

Applied to AMM-based DEXs, the batch-auction mechanism would:

(i) Collect all orders submitted within a block's time window.

(ii) Compute a single clearing price that maximises trading volume subject to the AMM's invariant.

(iii) Execute all matched trades simultaneously at that price.

This eliminates within batch frontrunning and sandwich attacks, as no single transaction can be "inserted" before another within the same batch. However, it introduces new strategic considerations, like users may time their submissions relative to batch boundaries, and miners retain the ability to manipulate *which* batch a transaction is included in.

2.4 Related Protocols and Extensions

- **CoW Protocol (Coincidence of Wants)**: An off-chain solver network that attempts to match orders peer-to-peer before routing residual flow through on-chain AMMs. Solvers compete in a batch auction for the right to settle a given set of orders.
- **MEV Boost/Flashbots**: An off-protocol relay that allows searchers (MEV extractors) to submit *bundles* directly to block builders, bypassing the public mempool. This makes MEV "private" and reduces on-chain PGA style bidding wars but does not eliminate MEV itself.
- **SUAVE (Single Unifying Auction for Value Expression)**: A proposed architecture by Flashbots that decentralises the block-building role, aiming to make MEV extraction more competitive and its revenue more broadly shared.

2.5 Summary of Literature Gaps

Table 1 summarises the key works and the gaps our project addresses.

3 Formal Game Theoretic Model

We now develop a formal model of strategic interactions in a DEX ecosystem. The model is designed to be tractable while capturing the essential incentive misalignments identified in the literature.

Table 1: Summary of Related Works and Identified Gaps

Work	Focus	Contribution	Gap
Daian et al. (2019) [1]	MEV measurement & PGA modeling	Quantifies MEV; formalises PGA as continuous game; grim-trigger equilibrium	No protocol-level counterfactual; no batch auctions
Alipanahloo et al. (2024) [3]	Fairness in rollup sequencing	Fairness granularity metric; randomised sequencing policy	No strategic user/miner model; no batch-auction comparison
Budish et al. (2015) [2]	HFT arms race in traditional markets	Batch auctions eliminate latency rents	Not applied to blockchain/AMM setting; no miner-strategic model
This project	Game-theoretic comparison: continuous vs. batch	Three player Stackelberg model; comparative equilibrium analysis; simulation	—

3.1 Overview and Modelling Philosophy

We model the DEX ecosystem as a **three-player extensive-form game** with sequential moves. The game structure reflects the natural timing of events on a blockchain:

Stage 1. **User stage:** Users observe the current AMM state and submit signed transactions with chosen gas prices and trade parameters.

Stage 2. **Miner/validator stage:** The block producer observes the full mempool and chooses a transaction ordering (and possibly inserts own transactions) to maximise their objective.

Stage 3. **DEX settlement stage:** The protocol clears transactions according to its rules (continuous or batch), producing final balances.

This sequential structure is a **Stackelberg game**, users are *leaders* who commit to trades before the miner's ordering decision, while miners are *followers* who best respond to the observable mempool. The DEX protocol is a rule setter whose design choice (mechanism) is treated as a parameter, enabling comparative analysis.

3.2 State Space

Definition 3.1 (AMM State)

Let $\mathcal{S} = \mathbb{R}^2_{>0}$ denote the state space of a constant-product AMM holding two assets A and B. A state is a pair $(r_A, r_B) \in \mathcal{S}$ where r_A, r_B are the reserve quantities. The *spot price* of asset A in terms of B at state (r_A, r_B) is

$$P(r_A, r_B) = \frac{r_B}{r_A}.$$

After a trade of Δ_A units of A for B, the new state is $(r_A + \Delta_A,\ r_B - \Delta_B)$ where Δ_B satisfies the *constant-product invariant*:

$$(r_A + \Delta_A)(r_B - \Delta_B) = r_A \cdot r_B.$$

This gives the standard AMM output formula:

$$\Delta_B = \frac{r_B \cdot \Delta_A}{r_A + \Delta_A}.$$

3.3 Players

3.3.1 Player 1: The User Set

Definition 3.2 (Users)

Let $\mathcal{U} = \{u_1, u_2, \ldots, u_n\}$ be a finite set of *users*. Each user u_i is characterised by:

- A *desired trade*: $(\delta_i, \mathrm{dir}_i)$ where $\delta_i > 0$ is the input quantity and $\mathrm{dir}_i \in \{A \to B,\ B \to A\}$ is the trade direction.
- A *slippage tolerance*: $\tau_i \in [0, 1]$, the maximum acceptable fraction of price impact.
- A *private valuation*: $v_i > 0$, the value the user places on executing the trade at the *fair* AMM price (i.e., without adverse price impact).
- A *budget constraint* on gas: $g_i^{\max} \geq 0$.

Users are modelled as *price taking* with respect to the AMM (they take the current state as given) but *strategic* with respect to their gas bid and submission timing.

3.3.2 Player 2: The Miner/Validator

Definition 3.3 (Miner/Validator)

The *miner* M is a single strategic agent (representing the block producer) who:

- Observes the full *mempool* $\mathcal{P}$: the set of all pending signed transactions submitted by users.
- Has the unilateral power to choose any ordering $\sigma \in \mathrm{Perm}(\mathcal{P})$ of the pending transactions.
- May insert *own transactions* (e.g., frontrunning trades) at any position in the block.
- Earns revenue from (i) gas fees, (ii) OO fees from strategic ordering, and (iii) profit from own inserted trades.

Note on the miner model: In proof-of-work systems, the miner is the entity that solves the block puzzle whereas in proof-of-stake systems, this role is played by the validator/proposer. We use "miner" throughout for brevity.

3.3.3 Player 3: The DEX Protocol

Definition 3.4 (DEX Protocol)

The *DEX protocol* Π is a rule- etting entity (not a strategic player in the game-theoretic sense, but a *mechanism designer*) that specifies:

- The *clearing rule*: continuous (sequential) or batch (uniform price auction).
- The *fee structure*: trading fees $\phi \in [0, 1]$ charged on each trade.
- Optionally, a *maximum slippage* parameter $\bar{\tau}$ beyond which trades are automatically reverted.

The protocol's objective is to maximise *market efficiency* (total trading volume and price accuracy) and *user welfare* (minimising aggregate slippage).

3.4 Strategy Spaces

3.4.1 User Strategy Space

Definition 3.5 (User Strategy)

A (pure) strategy for user u_i is a tuple

$$s_i = (t_i,\ g_i,\ \delta_i,\ \tau_i^{\text{set}}) \in \mathcal{S}_i^U$$

where:

- $t_i \geq 0$: submission time (relative to block start).
- $g_i \in [0, g_i^{\max}]$: gas price bid (in Gwei).
- $\delta_i > 0$: trade size (may differ from desired size as a form of demand splitting).
- $\tau_i^{\text{set}} \in [0, \tau_i]$: on-chain slippage tolerance encoded in the transaction.

A *mixed strategy* for user u_i is a probability distribution $\mu_i \in \Delta(\mathcal{S}_i^U)$ over pure strategies.

Remark 3.1. In the continuous trading setting, users have an incentive to submit with high gas prices to gain priority over both other users and (defensive) bots. In the batch auction setting, gas prices no longer determine within-batch ordering, eliminating this particular strategic dimension.

3.4.2 Miner Strategy Space

Definition 3.6 (Miner Strategy)

Let $\mathcal{P} = \{x_1, \ldots, x_k\}$ denote the set of pending transactions observed by the miner. A (pure) strategy for the miner is a pair

$$s_M = (\sigma,\ \mathcal{I}) \in \mathcal{S}^M$$

where:

- $\sigma : \{1, \ldots, k\} \to \{1, \ldots, k\}$ is a permutation specifying the order in which user transactions are executed.
- $\mathcal{I} = \{y_1, \ldots, y_m\}$ is a (possibly empty) set of transactions inserted by the miner, each with a specified position in the block.

The miner's *feasibility constraint* requires the total gas used by all transactions to not exceed the block gas limit $G^{\max}$.

We distinguish three canonical miner strategies:

M1. Honest ordering: σ orders transactions by decreasing gas price; $\mathcal{I} = \emptyset$. This is the "default" miner behaviour.

M2. Frontrunning: The miner inserts own transaction(s) before a detected profitable user transaction to capture the price impact.

M3. Sandwich attack: The miner inserts a buy transaction *before* and a sell transaction *after* a large user trade to profit from the price impact it causes.

3.4.3 Sandwich Attack: Formal Specification

Definition 3.7 (Sandwich Strategy)

Given a target user trade $x_i = (\delta_i, A \to B)$, a sandwich strategy is $(\sigma, \mathcal{I})$ where $\mathcal{I} = \{y^+, y^-\}$ with:

- y^+: miner sells ϵ units of A at state (r_A, r_B), shifting the price to $P'(r_A, r_B) < P(r_A, r_B)$.
- User trade x_i executes at the worse price P', receiving $\Delta'_B < \Delta_B$.
- y^-: miner buys ϵ units of A back at the post-user-trade state, netting a profit.

The miner's sandwich profit is:

$$\pi_M^{\text{sandwich}} = \Delta_B^{y^+} - \Delta_B^{y^-} > 0.$$

3.5 Payoff Functions

3.5.1 User Payoff

Definition 3.8 (User Payoff)

Let $\hat{\Delta}_B^i$ be the actual output of user u_i's trade under the miner's chosen ordering, and let Δ_B^i be the "fair" output (at the state the user expected when submitting, ignoring any adversarial ordering). The payoff to user u_i is:

$$\pi_i^U = v_i \cdot \frac{\hat{\Delta}_B^i}{\Delta_B^i} - g_i \cdot c_{\text{gas}} - \mathbb{1}[\text{reverted}] \cdot g_i^{\text{used}} \cdot c_{\text{gas}}$$

where v_i is user's own valuation and c_{gas} is the ETH-per-Gwei conversion, and the indicator term charges the user for gas consumed even if the transaction reverts (as in Ethereum).

The *slippage* experienced by user u_i is:

$$\text{slip}_i = 1 - \frac{\hat{\Delta}_B^i}{\Delta_B^i} \in [0, 1].$$

3.5.2 Miner Payoff

Definition 3.9 (Miner Payoff)

The miner's total payoff from mining a block with strategy $(\sigma, \mathcal{I})$ is:

$$\pi_M = \underbrace{\sum_{i=1}^{k} g_{\sigma(i)} \cdot \text{gas}_{\sigma(i)}}_{\text{gas fees}} + \underbrace{\sum_{y_j \in \mathcal{I}} \pi_M^{(j)}}_{\text{OO / MEV}} + B_{\text{block}}$$

where B_{block} is the fixed block reward, $g_{\sigma(i)}$ is the gas price of the i-th ordered transaction, $\text{gas}_{\sigma(i)}$ is its gas consumption, and $\pi_M^{(j)}$ is the profit from the j-th inserted transaction.

In a **sandwich attack**, the miner's incremental MEV is:

$$\pi_M^{\text{MEV}} = \Delta_B^{y^+} - \Delta_B^{y^-} = \frac{r_B \epsilon}{r_A + \epsilon} - \frac{(r_B - \Delta_B^{y^+} - \hat{\Delta}_B^i)\,\epsilon}{(r_A + \delta_i)}$$

Here, we have simplified by assuming the miner uses the same ϵ for both legs of the sandwich.

3.5.3 Protocol Payoff / Social Welfare

Definition 3.10 (Social Welfare)

The *social welfare* of an outcome is:

$$W = \sum_{i=1}^{n} \pi_i^U + \pi_M - \text{deadweight loss}$$

where the deadweight loss arises from trades that are reverted due to exceeded slippage tolerances or from resources expended in the PGA bidding process that do not create value (e.g., gas burned by losing bids).

Since π_M includes MEV *extracted from* users, it is more illuminating to report **user welfare** separately:

$$W^U = \sum_{i=1}^{n} \pi_i^U.$$

3.6 Information Structure

Definition 3.11 (Information Sets)

- **Users:** Each user u_i observes the current AMM state (r_A, r_B) and the current mempool $\mathcal{P}^{-i}$ (transactions of others) with a latency $\Delta_i^U \geq 0$. Users do *not* observe the miner's ordering decision before submitting.
- **Miner:** The miner observes the *complete* mempool $\mathcal{P}$ before making its ordering decision. This is the miner's fundamental information advantage.
- **Protocol:** The protocol has no private information; it mechanically applies the clearing rule to whatever ordered list of transactions it receives.

3.7 Equilibrium Concepts

3.7.1 Nash Equilibrium

Definition 3.12 (Nash Equilibrium)

A strategy profile $(s_1^*, \ldots, s_n^*, s_M^*)$ is a *Nash equilibrium* (NE) if no player can strictly increase their payoff by unilaterally deviating.

$$\pi_i^U(s_i^*, s_{-i}^*) \geq \pi_i^U(s_i, s_{-i}^*) \quad \forall\, s_i \in \mathcal{S}_i^U,\ \forall\, i$$

$$\pi_M(s_M^*, s_{-M}^*) \geq \pi_M(s_M, s_{-M}^*) \quad \forall\, s_M \in \mathcal{S}^M.$$

3.7.2 Stackelberg Equilibrium

Definition 3.13 (Stackelberg Equilibrium)

In the *Stackelberg formulation*, users are **leaders** and the miner is the **follower**.

1. Users simultaneously choose strategies $(s_1, \ldots, s_n)$, which are observed by the miner.
2. The miner best-responds:

$$s_M^*(s_{-M}) = \arg\max_{s_M \in \mathcal{S}^M} \pi_M(s_M;\, s_1, \ldots, s_n).$$

3. A *Stackelberg equilibrium* is a profile $(s_1^*, \ldots, s_n^*,\ s_M^*)$ such that users play best-responses given the miner's best-response correspondence:

$$s_i^* \in \arg\max_{s_i} \pi_i^U(s_i, s_{-i}^*;\ s_M^*(s_1^*, \ldots, s_i, \ldots, s_n^*)).$$

Remark 3.2. The Stackelberg formulation captures the reality that users submit transactions *before* the miner selects an ordering, but *after* the miner has set up infrastructure to exploit known user patterns. Users who are aware of the miner's strategy must choose their gas bid, trade size, and slippage tolerance defensively.

3.7.3 Repeated Game and Cooperation

Drawing on the cooperative equilibrium observed by Daian et al. among PGA bots, we also consider a **repeated game** formulation where the same block is a single stage of a game repeated across many blocks.

Definition 3.14 (Repeated Game Payoff)

In the repeated game with discount factor $\beta \in (0, 1)$, player i's payoff from stage-game payoff sequence $(\pi_i^t)_{t=0}^{\infty}$ is:

$$\Pi_i = (1 - \beta) \sum_{t=0}^{\infty} \beta^t \pi_i^t.$$

The Folk Theorem guarantees that any feasible and individually rational payoff profile can be sustained in a subgame-perfect equilibrium of the infinitely repeated game, provided β is sufficiently close to 1. This provides theoretical grounding for the cooperative equilibria observed empirically.

3.8 Priority Gas Auction Sub-Game

When multiple users (or bots) compete for the same block position, their interaction is modelled as a PGA sub-game, formalised following Daian et al.:

Definition 3.15 (PGA Sub-Game)

A *Priority Gas Auction* among n competitors for a single block position is a sequential game with:

- **Continuous time:** Bids can be submitted at any time $t \in [0, T)$ where $T \sim \mathrm{Exp}(\lambda)$ is the random block-mining time.
- **All-pay:** The losing player pays $\ell(\$b_{\text{last}})$ where $\$b_{\text{last}}$ is their final gas bid and $\ell(\cdot)$ is a loss function ($\ell(b) < b$, partial all-pay).
- **Minimum raise constraint:** Any new bid must exceed the player's previous bid by at least a factor $(1 + \iota)$, where $\iota = 0.125$ under Parity client defaults.
- **Imperfect information:** Player i observes competitor bids with latency $\Delta_i \geq 0$.

Theorem 3.1 (Cooperative Nash Equilibrium in PGAs [1])**.** *For parameters consistent with Ethereum PGAs (exponential block time with $\lambda^{-1} = 15\,s$, minimum raise $\iota = 0.125$, mean inter-bid interval $\delta = 0.4\,s$, latency $\Delta_i = 0.1\,s$), there exists a* grim trigger Nash equilibrium *in which two competing players alternate bids at minimum-raise increments, and any deviation is met with an immediate bid to the maximum profitable level.*

3.9 The Batch Auction Extension

Definition 3.16 (Batch Auction Mechanism)

Under a *batch auction* clearing rule, the DEX protocol:

(i) Collects all transactions submitted within block interval $[0, T)$.

(ii) Determines a *uniform clearing price* P^* that maximises the volume of matched buy and sell orders subject to the AMM invariant.

(iii) Executes all matched trades simultaneously at P^*.

Formally, P^* solves:

$$P^* = \arg\max_P \min\left(\sum_{i:\,\mathrm{dir}_i = A \to B,\, v_i \geq P} \delta_i, \sum_{j:\,\mathrm{dir}_j = B \to A,\, v_j \leq P} \delta_j \right).$$

Proposition 3.1 (Elimination of Within-Batch Frontrunning)**.** *Under the batch auction mechanism, for any two transactions x_i, x_j submitted within the same batch, the miner's choice of ordering σ over $\{x_i, x_j\}$ does not affect their individual execution prices. Consequently, no sandwich attack is profitable within a batch.*

Proof sketch. Since all trades within a batch execute at the uniform clearing price P^*, the miner has no mechanism to insert a transaction between x_i and x_j that would shift the price at which either trade executes. Any miner-inserted transaction y within the batch is also subject to the same clearing price P^*, eliminating the asymmetric price impact that makes sandwiching profitable. □

4 Comparative Analysis: Continuous vs. Batch Clearing

4.1 Equilibrium Outcomes Under Continuous Trading

Under continuous AMM trading, the Stackelberg equilibrium is characterised by the following dynamics:

(i) **Miner best response:** For any user transaction with trade size δ_i large enough that the slippage $\text{slip}_i > 0$, the miner's dominant strategy (given full mempool visibility) is to sandwich the trade if:

$$\pi_M^{\text{sandwich}}(\delta_i) > \text{cost of inserted gas.}$$

(ii) **User best response:** Anticipating sandwiching, rational users set low slippage tolerances τ_i^{set} and split large trades into smaller chunks—both of which reduce sandwich profitability but also reduce trading efficiency.

(iii) **Gas price war:** Multiple users competing for priority face a PGA sub-game (Definition 3.15) over gas prices, driving bids up and wasting value in the all-pay component.

4.2 Equilibrium Outcomes Under Batch Auctions

Under batch-auction clearing:

(i) **Elimination of within-batch MEV:** By Proposition 3.1, sandwich attacks are infeasible within a batch. Frontrunning across batches (i.e., the miner choosing *which* batch to include a transaction in) remains possible but is coarser and less profitable.

(ii) **User strategy simplification:** Users need not set defensive slippage limits to protect against within batch ordering; their only strategic variable becomes submission timing relative to batch windows.

(iii) **Residual miner MEV:** The miner retains the ability to delay or accelerate a transaction's batch membership. However, the value of this is bounded by the price movement between consecutive batches, which is smaller than within-transaction slippage under continuous trading for reasonable batch durations.

4.3 Welfare Comparison

Table 2 summarises the theoretical welfare predictions.

Table 2: Predicted Welfare Outcomes: Continuous vs. Batch Clearing

Metric	**Continuous AMM**	**Batch Auction**
User slippage (avg.)	High (sandwich attacks)	Low (uniform price)
Miner MEV revenue	High	Low/Moderate
Gas fee waste (PGAs)	High	Low (no priority race)
Trade execution latency	Low (immediate)	Higher (batch window)
Price discovery efficiency	Continuous	Batch interval resolution
Frontrunning vulnerability	High	Low (within batch)
Consensus security risk	High (OO fees)	Reduced

5 Code Introduction

The simulation models a blockchain network in which users submit token swap transactions to a shared public mempool, miners compete to produce blocks, and a constant product AMM DEX settles each swap according to one of two clearing rules. Under **continuous clearing**, transactions inside a block are executed sequentially in the order the miner chooses; each swap immediately updates the AMM reserve state before the next transaction is processed. Under **batched clearing**, all transactions selected for a block are executed simultaneously at a single uniform rate derived from the block's aggregate demand.

The key distinction between the two modes is the opportunity they offer for MEV extraction. In continuous mode an "attacker" miner can insert its own transactions between user transactions and profit from the induced price movement; in batched mode "attacker" miner can insert its own transactions but every transaction in the block is processed at the same rate, eliminating the within block ordering advantage.

Code simulation does not provide the complete strategic and game theoretic view as in the theoretical claims. Various assumptions and simplification were made due to time and resource constraints and a very simplified version is being simulated.

The codebase comprises ten Python modules. Figure 5 shows the dependency structure.

Module Dependency Overview

simulation.py $\longrightarrow$ blockchain.py $\longrightarrow$ peer.py, mempool.py, dex.py

mempool.py $\longrightarrow$ pga.py $\longrightarrow$ txn.py, dex.py

peer.py $\longrightarrow$ block.py, blocktree.py

block.py $\longrightarrow$ txn.py

Throughout this report the term *peer* refers to any participant in the network. Peers are divided into two roles:

- **Miners** — participants who both generate transactions *and* produce blocks. Miners are further classified as *honest* (order transactions by gas price only) or *attacker* (insert own transactions to extract MEV). All miners are assigned *low latency*, reflecting the real world observation that block producers invest in fast network infrastructure.
- **Users** — participants who only generate transactions; they never mine blocks. Users are subdivided into *normal users* (high latency) and *bots* (some low latency, some high latency). Bots monitor the mempool for PGA opportunities; normal users do not.

6 Module Descriptions

6.1 txn.py: Transaction

Every transaction is uniquely identified by the pair (`peer_id`, `nonce`). Using a nonce rather than a random transaction ID allows the same logical transaction to be *replaced* in the mempool by a higher gas price version, which is the mechanism underlying Priority Gas Auctions.

Attributes

- peer_id — the identifier of the peer that created the transaction.
- nonce — a per peer counter that, together with peer_id, uniquely identifies a transaction slot. A peer may submit a new transaction with the same nonce but a higher gas price to replace a pending one (PGA bidding).
- type — either "swap" or "transfer". In the present simulation *all* transactions are swaps; the transfer type is retained for extensibility.
- Swap-specific fields: dex_id (which DEX to use), idx1 and idx2 (token indices), amt (input amount of idx1), tlr (slippage tolerance τ), and min_out (minimum acceptable output, encoding the revert condition $\hat{\Delta}_B \geq \texttt{min_out}$).
- gas_price — the fee per unit of gas the creator offers. Miners use this to prioritise transactions; bots raise it during PGAs.
- timestamp — simulation time at which the transaction was created.
- code — a randomly drawn integer representing the gas units consumed when the transaction is executed. Swap transactions draw from Uniform$(8, 20)$ to reflect higher computational cost; transfer transactions draw from Uniform$(1, 3)$. The block gas limit is enforced against cumulative code values when assembling a block.

Design note

The min_out field is the on-chain slippage guard. If the actual output of a swap at execution time is below min_out, the transaction is treated as reverted, the user is charged gas but receives no tokens. This is the primary user side defence against sandwich attacks.

6.2 dex.py: Automated Market Maker

The DEX class implements a two-token constant-product AMM. The simulation initialises a single DEX instance whose reserves (r_0, r_1) are supplied as command-line arguments.

Attributes

- tokens — list of current reserve amounts $[r_0, r_1]$. This is the mutable state that changes with every executed swap.
- initial_tokens — list of the reserves at initialisation. Stored separately so that the initial state can be recovered or referenced after the simulation.

Key methods

- get_swap(idx1, idx2, amt) — *non-mutating* preview: computes and returns

$$\Delta_{\texttt{idx2}} = \frac{r_{\texttt{idx2}} \cdot \texttt{amt}}{r_{\texttt{idx1}} + \texttt{amt}}$$

  without updating the reserves. Used by the PGA module to evaluate profit before committing.

- swap(idx1, idx2, amt) — *mutating* execution: applies the same formula and updates tokens in place. Called during block execution in the simulation engine.
- get_relative_price(idx1, idx2) — returns the current spot price of idx1 in terms of idx2: $P = r_{\texttt{idx2}}/r_{\texttt{idx1}}$ when idx1 == 0, and 1.0 otherwise (since token 1 is the unit of account in this simulation).
- get_curr_state() — returns a reference to tokens, used by the simulation to record reserve snapshots.

Design note

Separating get_swap (read-only) from swap (mutating) is critical for correctness: the PGA profit calculation and the attacker's insertion logic both need to query the DEX state without disturbing it, whereas block execution must update the state in the order transactions are processed.

6.3 block.py: Block

The Block class represents a single block on the blockchain.

Attributes

- id — a randomly generated 64-bit integer serving as the block's unique identifier (analogous to a block hash).
- prev_blk_id — the id of the parent block, establishing the chain linkage. The genesis block uses the sentinel value -2 as its own prev_blk_id.
- timestamp — the simulation time at which mining of this block completed.
- txns — list of Transaction objects included in the block. The ordering reflects the miner's strategy: honest miners order by gas price; attacker miners may insert own transactions at chosen positions.
- creator_id — the peer_id of the miner who produced the block. Used to attribute the block reward and to identify attacker-mined blocks in analysis.
- balance — a dictionary mapping each peer's peer_id to a two-element list [bal_token0, bal_token1] recording that peer's token holdings *after* all transactions in this block have been applied.

6.4 blocktree.py: Block Tree

Each peer maintains an independent BlockTree instance representing its local view of the blockchain, including any active forks.

Data structures

- tree — a list of dictionaries indexed by chain depth (level). tree[d] maps the id of every block at depth d to its parent's id. Level 0 contains only the genesis block.
- id_level_map — a flat dictionary mapping block ids to their depth. This supports $O(1)$ depth lookup and is necessary for efficient orphan resolution.

- orphan_blocks — a dictionary of blocks received before their parent was known, keyed by the orphan's id and valued by the expected parent id. When a new block is successfully inserted, the tree checks whether any orphan now has its parent present and resolves the dependency recursively.

Key methods

- add_block(id, prev_blk_id) — inserts the block at depth $d_{\text{parent}} + 1$. If multiple blocks share the same depth, all are retained in tree[d], faithfully representing a fork. Orphan resolution runs after every successful insertion.
- get_longest_chain_leaves() — returns all block ids at the deepest level of tree. When multiple tips exist (a fork), the simulation resolves ties by preferring the tip with the earliest timestamp.
- get_active_blocks() — counts all blocks that lie on at least one path from the deepest level back to genesis. This distinguishes blocks on the canonical chain (or any competing fork) from true orphans, providing a measure of chain utilisation.

6.5 peer.py: Peer

The Peer class represents a single participant in the network.

Attributes

- peer_id — unique integer identifier.
- type — one of "honest", "attacker", or "user".
- gen — the genesis Block instance for this peer's local chain. Its id is initialised to -1 (a sentinel distinct from the -2 used in the block tree root).
- tree — the peer's BlockTree, initialised with root -1 to match the genesis block's id.
- prev_blk — the id of the block on which this peer is currently mining. Set by START_MINING and used by FINISH_MINING to detect stale mining events (i.e. the chain tip moved while this peer was mining).
- latency — one of "low (bot)", "mid (bot)", or "high". All miners have low latency. Users are either high-latency (normal users) or bot-latency (low or mid).
- nonce — a randomly initialised integer that is incremented each time the peer creates a new transaction. Together with peer_id it uniquely identifies each transaction slot.

Peer taxonomy

Type	Role	Latency
"honest"	Mines blocks; orders txns by gas price; no insertion	Low
"attacker"	Mines blocks; inserts own txns for MEV	Low
"user"	Generates transactions only; never mines	High (normal) or bot

Table 3: Peer types in the simulation.

Design note

Implementation has a single *global* mempool is maintained by the Blockchain object (see §6.8). This better reflects the reality that all peers on a well-connected network rapidly converge to a consistent view of pending transactions.

6.6 pga.py — Priority Gas Auction

The PGA class models a single Priority Gas Auction centred on one *honest user transaction* (the target being front-run or sandwiched).

Background

When a profitable user swap appears in the mempool, competing bots race to be included *before* (or around) it by repeatedly resubmitting a higher-gas version of their own transaction. This is the on chain Priority Gas Auction documented by Daian et al. [1]. Each PGA is tracked as a separate object so that multiple simultaneous auctions can coexist in the mempool.

Attributes

- honest_txn — the original user transaction that triggered this PGA. It identifies the target trade.
- peer_id, txn_nonce — identifiers of the honest transaction, used to link the PGA to its trigger.
- min_inc — the minimum fractional increment by which a new bid must exceed the current highest bid. Set to 0.125 (12.5%), matching the Parity client default studied empirically by Daian et al. [1].
- pga_state — a list of transactions (one per bidding peer), kept sorted in descending order of gas_price. The first element is always the current highest bidder.
- txn_ids — a dictionary mapping peer_id to the nonce of that peer's current bid in this PGA. Used for $O(1)$ duplicate detection.

Key methods

- get_max_profit(dex) — computes the maximum sandwich profit attainable on the honest transaction at the current DEX state, i.e. the optimal front-run size ϵ^*. Derived by solving the condition that the user's minimum output constraint is exactly met after the front-run:
$$\epsilon^* = \frac{-\texttt{amt} + \sqrt{\texttt{amt}^2 + \dfrac{4\, r_{\texttt{idx1}}\, r_{\texttt{idx2}}\, \texttt{amt}}{\texttt{min_out}}}}{2} - r_{\texttt{idx1}}.$$
Returns $(-1, -1, -1)$ if no profitable front-run exists (i.e. the honest transaction's min_out is already tighter than what the current state allows).
- get_pga_state(peer_id) — returns False if the querying peer is already at the top of this PGA (i.e. is the current highest bidder), and True otherwise. Bots use this to decide whether to raise their bid.
- update_pga_state(peer_id, nonce, txn) — adds or updates a bid in the auction. If the peer is new, the transaction is appended and the list is re sorted. If the peer already has a bid, the new gas price must exceed the old one by at least `min_inc`; otherwise the update is ignored. After any valid update the list is re sorted by gas_price.

6.7 mempool.py — Mempool

The Mempool class is the global shared pool of pending transactions. It is owned by the Blockchain object and is visible to all peers, reflecting the public broadcast model of Ethereum's gossip network.

Attributes

- txns — list of all currently pending Transaction objects.
- txn_id_map — a dictionary keyed by $(\texttt{peer_id}, \texttt{nonce})$ pointing to the Transaction currently occupying that slot. Supports $O(1)$ lookup for replacement (gas-price escalation) and duplicate detection.
- min_inc — the minimum fractional gas-price increase required to replace an existing transaction with the same $(\texttt{peer_id}, \texttt{nonce})$ key. Defaults to 0.125.
- pga_list — a list of active PGA objects. A new PGA is created whenever a user swap appears that a bot considers profitable; it is removed when its target transaction leaves the mempool.

Key methods

- add_txn(txn) — adds a transaction to the mempool. If a transaction with the same $(\texttt{peer_id}, \texttt{nonce})$ already exists, the new one replaces it only if $\texttt{gas_price}_{\text{new}} \geq (1 + \texttt{min_inc}) \cdot \texttt{gas_price}_{\text{old}}$. This enforces the minimum-raise rule and prevents cheap spam replacements.
- remove_txn(txn) — removes a transaction and cleans up any associated PGA entries. If the removed transaction is the trigger of a PGA, that PGA is dissolved. If it is the top bid in a PGA's state list, the bid is popped and the PGA continues with the next-highest bidder; if the list becomes empty the PGA is dissolved.

- get_mempool_info() — returns shallow copies of txns and txn_id_map, used by miners to snapshot the pending transaction set when assembling a block.

6.8 blockchain.py: Blockchain

The Blockchain class is the global state container for the simulation. It holds all peers, the DEX instance, the shared mempool, and the P2P network topology.

Attributes

- peers — the complete list of Peer objects (miners and users).
- dex — a list of DEX objects; in the current simulation exactly one DEX is used.
- mempool — the single shared Mempool instance. Placing the mempool here rather than inside each peer reflects the global mempool model..
- topology — a dictionary mapping each peer_id to the set of its neighbours' ids. Generated by create_topology().

Key methods

- create_topology() — generates a random connected peer-to-peer network. Each peer is assigned between 3 and 6 neighbours. Connectivity is verified via depth-first search (check_connected); if the graph is disconnected the topology is reset and regenerated from scratch.
- check_connected(topology) — runs a DFS from peer 0 and returns True only if every peer is reachable.
- get_peer(id) — linear-scan lookup returning the Peer with the given id, or None if not found.
- add_dex(dex) — registers a DEX instance, appending it to self.dex.

6.9 simulation.py — Event-Driven Simulation Engine

The simulation engine implements a classic discrete event simulator using a min-heap priority queue (heapq) sorted by scheduled event time. All times are in milliseconds. The simulation advances by repeatedly popping the earliest event, updating current_time, and dispatching to the appropriate handler.

Initialisation

The create_blockchain function builds the network before the simulation loop starts:

1. Creates n user peers partitioned by latency class: a fraction are high-latency normal users; the remainder are low- or mid-latency bots.
2. Appends one "honest" miner and one "attacker" miner, both at low latency.
3. Calls create_topology() to wire up the P2P graph.
4. Initialises a two-token DEX with reserves (r_0, r_1) supplied by the user.

5. Distributes initial token balances uniformly: each peer receives $r_0/(10n)$ of token 0 and $r_1/(10n)$ of token 1.

Event types

Table 4: Simulation event types and their handlers.

Event	**Description**
CLEAR_MEMPOOL	Periodically removes the oldest transactions from the mempool to prevent unbounded growth. This simulates the natural expiry of stale pending transactions on live networks.
GENERATE_TRANSACTION	A *user* (not a bot and not a miner) generates a swap transaction. The swap amount is drawn from a uniform distribution. The inter-arrival time between consecutive transactions from the same user is drawn from an exponential distribution with mean T_{tx} (a simulation parameter). The transaction is added to the global mempool and to any relevant PGA. The next GENERATE_TRANSACTION event for this user is scheduled immediately.
LOOKUP_MEMPOOL	A *bot* inspects the global mempool for existing PGAs. It selects one at random from the available PGAs and, if it is not already the top bidder, raises its bid by at least min_inc and submits a new transaction. Bots with lower latency observe the mempool more frequently (shorter inter-lookup intervals), giving them a competitive advantage in PGAs.
START_MINING	A miner (honest or attacker) is randomly selected proportional to number. The selected miner identifies the tip of its longest chain (earliest timestamp tie break) and schedules a FINISH_MINING event after a draw from uniform distribution with mean as target mean inter-block time (12 seconds, matching Ethereum).

Event	Description
FINISH_MINING	The block is assembled and broadcast. The event is treated as stale and discarded if the miner's chain tip has changed since START_MINING. Block assembly depends on the clearing mode: • **Continuous mode**: transactions are drawn from the mempool (sorted by gas price), subject to the block gas limit. An attacker miner scans for PGA opportunities and inserts its own transaction *before* the target user transaction. Each included transaction is executed against the DEX in order; the DEX state is updated after every swap so that each subsequent transaction executes at the current (post-previous-swap) price. • **Batched mode**: the same set of transactions is selected, but all swaps are executed at a single *uniform clearing rate*. The clearing rate is computed from the aggregate demand of the entire batch, so no single transaction can benefit from a more favourable ordering. An attacker miner can still attempt to insert its own transaction into the batch, but since all transactions clear at the same rate the attacker gains no price-impact advantage within the block. In both modes the miner collects the block reward plus gas fees. The Block.balance snapshot is computed and stored. The block is then broadcast to all neighbours with per-link delay.
RECEIVE_BLOCK	A peer receives a new block. The block is added to the peer's local BlockTree. The peer then forwards the block to all its neighbours (excluding the sender) with per-link delay.

Metrics recorded

The simulation records the following welfare metrics over all mined blocks on the canonical chain:

- **User slippage** $\overline{\text{slip}}$: mean fractional deviation between actual and fair swap output for each user type (normal user, low-latency bot, high-latency bot).
- **Transaction execution latency**: wall clock delay from transaction submission to inclusion in a block, broken down by user type.
- **Gas fee waste**: total gas consumed by transactions that reverted due to the min_out constraint being violated.
- **Miner payoffs**: total gas fees and MEV profit collected by the honest miner and the attacker miner respectively.
- **Expected user payoff**: aggregate token gain or loss for each user type across the

simulation.

7 Interaction Between Modules

This section describes how the modules interact during the key simulation events to give a complete picture of the data flow.

7.1 Transaction Generation and PGA Formation

When a GENERATE_TRANSACTION event fires for a user peer:

1. A Transaction object is constructed with a fresh nonce, a random amount, a computed min_out (based on the current DEX price and the user's slippage tolerance), and a randomly drawn gas price.
2. Mempool.add_txn inserts the transaction.
3. If the transaction's min_out leaves room for a profitable front-run (determined via PGA.get_max_profit), a new PGA object is created and appended to Mempool.pga_list.

When a LOOKUP_MEMPOOL event fires for a bot peer:

1. The bot samples a random PGA from Mempool.pga_list.
2. It calls PGA.get_pga_state to check if it is already the top bidder.
3. If not, it creates a new Transaction with the same nonce as its previous bid (or a new nonce if it has no current bid) at a gas price of $(1 + \texttt{min_inc})$ times the current top bid.
4. The new transaction is submitted via Mempool.add_txn and PGA.update_pga_state.

7.2 Block Assembly in Continuous Mode

During FINISH_MINING for an attacker miner in continuous mode:

1. The miner snapshots the mempool via Mempool.get_mempool_info().
2. It sorts pending transactions by gas price (descending) and greedily fills the block up to the gas limit (summing Transaction.code values).
3. For each PGA in Mempool.pga_list, the miner checks whether it is the top bidder. If so, it extracts the target user transaction's position and inserts its own front-run transaction immediately before it.
4. Transactions are executed one by one against the live DEX state (DEX.swap), in the miner's chosen order. Each swap updates the reserves before the next transaction.
5. The Block.balance snapshot is populated.

7.3 Block Assembly in Batched Mode

The same selection step is followed, but instead of sequential execution:

1. All swap amounts are aggregated to compute a net demand for each token direction.

2. A single uniform clearing rate is derived from the aggregate demand and the current DEX reserves, applying the constant-product formula once to the combined input.
3. Every transaction receives an output proportional to its input at this uniform rate, regardless of its position in the list.
4. The attacker's inserted transaction is subject to the same clearing rate; it cannot exploit a favourable position.

7.4 Block Propagation

After a block is created:

1. The creating miner calls BlockTree.add_block on its own tree.
2. Each receiving peer calls BlockTree.add_block on its own tree and schedules further propagation to its own neighbours (excluding the sender), again using per-link latency.

8 Simplifications and Limitations

The following simplifications were made relative to the full model in order to keep the simulation tractable:

S1. Single DEX. Only one two-token AMM is present. Multi-DEX arbitrage, an important real world MEV source is not modelled.

S2. All transactions are swaps. Transfer transactions are defined in txn.py for completeness but are never generated by the simulation. This focuses the welfare analysis entirely on AMM-related MEV.

S3. Global mempool. All peers see the full mempool instantly. In practice, mempool propagation takes time and different nodes see different pending transactions. The latency difference between bots and normal users is captured through the LOOKUP_MEMPOOL frequency rather than through delayed mempool visibility.

S4. No block validation by receivers. Receiving peers add blocks to their tree without re-executing transactions against the balance snapshot. Validation is the miner's sole responsibility in this model.

S5. Simplified batched clearing. The uniform clearing rate is computed as a single aggregate swap rather than a true batch auction that maximises matched volume subject to individual reservation prices. This is a conservative simplification that underestimates the price-improvement benefit of batching for individual traders.

References

[1] Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. Flash boys 2.0: Frontrunning, transaction reordering, and consensus instability in decentralised exchanges. *arXiv preprint arXiv:1904.05234*, 2019.

[2] Eric Budish, Peter Cramton, and John Shim. The high-frequency trading arms race: Frequent batch auctions as a market design response. *The Quarterly Journal of Economics*, 130(4):1547–1621, 2015.

[3] Zeinab Alipanahloo, Kaiwen Zhang, and Emmanuel Awosika. Quantifying fairness granularity as a fair ordering policy towards MEV mitigation for rollups. In *2024 IEEE International Conference on Blockchain (Blockchain)*, pages 209–218. IEEE, 2024.